

**3x Raspberry Pi 4 8GB | Ubuntu 22.04.5 LTS | FRRouting | MPLS |
IPSec**

TABLE OF CONTENTS

Chapter 01 -- Network Architecture Overview

IP scheme, topology, what each Pi does

Chapter 02 -- Before You Begin

Hardware checklist, internet requirement, SSH keys

Chapter 03 -- Station 1 — First Boot Setup

Display setup, password, static IP, SSH

Chapter 04 -- Station 2 — First Boot Setup

Headless after Station 1 is SSHable

Chapter 05 -- Station 3 — First Boot Setup

Final Pi on the network

Chapter 06 -- Laptop Preparation

Ubuntu dual boot, SSH keys, NTP server setup

Chapter 07 -- Network Verification

All 4 devices pinging, SSH from laptop to all Pis

Chapter 08 -- Chrony NTP Synchronisation

Time sync across all devices

Chapter 09 -- System-Wide Package Preparation

Common packages, kernel modules, sysctl

Chapter 10 -- FRRouting Installation

Install and configure FRR on all 3 stations

Chapter 11 -- OSPF Configuration

OSPF adjacency between all 3 Pis

Chapter 12 -- BGP Configuration

iBGP sessions, MP-BGP for VPN routes

Chapter 13 -- MPLS LDP Configuration

Label distribution, MPLS forwarding table

Chapter 14 -- VRF Configuration

MISSION_CONTROL and ADMIN virtual routing tables

Chapter 15 -- Network Namespace CE Simulation

Simulating CE-A and CE-B customer sites

Chapter 16 -- MPLS-TE with RSVP-TE

Traffic engineering LSPs and bandwidth reservation

Chapter 17 -- strongSwan IPSec Tunnels

Encrypted SD-WAN overlay tunnels between Pis

Chapter 18 -- iPerf3 Traffic Generation

Application traffic flows and jitter measurement

Chapter 19 -- Full Simulation Verification

End-to-end test of the complete topology

Chapter A -- Troubleshooting Reference

Common errors and exact fixes

Chapter B -- Quick Command Reference

All essential commands in one place

01 Network Architecture Overview

Before touching a single command, understand the topology you are building. Everything in this guide follows from this architecture.

The Three Pis and Their Roles

Device	Hostname	Static IP	MPLS Role	Key Software
Pi 1	station1	192.168.1.10	CE-A + PE-1	FRRouting, strongSwan, Telegraf
Pi 2	station2	192.168.1.20	CE-B + PE-2	FRRouting, iPerf3, Telegraf
Pi 3	station3	192.168.1.30	P core router	FRRouting, pmacct, NetFlow
Laptop	deca-brain	192.168.1.1	Inference node	Prometheus, ML, LLM, FastAPI

Physical Network

All four devices connect to the same gigabit switch via ethernet. The switch is NOT connected to the internet during the demo -- but MUST be connected during the setup phase for package installation.

```
Physical connections:
station1 ---eth0---+
station2 ---eth0---+--- [Gigabit Switch] ---eth0--- Laptop
station3 ---eth0---+

During setup: Switch connects upstream to router (internet)
During demo: Upstream cable unplugged -- air-gapped LAN only
```

Simulated MPLS Topology

FRRouting runs on each Pi's main network namespace. The OSPF/BGP/LDP routing protocols peer across the physical ethernet interfaces. CE sites are simulated using Linux network namespaces on Station 1 and Station 2.

```
Simulated MPLS path:

[CE-A namespace] <--> [PE-1 / station1] <--> [P core / station3] <--> [PE-2 / station
2] <--> [CE-B namespace]
10.10.1.1/30          192.168.1.10          192.168.1.30          192.168.1.20
10.10.2.1/30

Loopback IPs (router-IDs and LDP transport):
station1 lo: 10.1.1.1/32
station2 lo: 10.1.2.1/32
```

```

station3 lo: 10.1.3.1/32

VRFs on station1 (PE-1) and station2 (PE-2):
MISSION_CONTROL  -- RD 65001:100  RT 65001:100
ADMIN            -- RD 65001:200  RT 65001:200

BGP AS numbers:
All PEs and P:  AS 65001 (iBGP)
CE-A, CE-B:     AS 65000 (eBGP into PE)

```

Build Sequence Summary

You must follow this sequence exactly. Each phase depends on the previous one being stable.

Phase	What You Build	Verify Before Moving On
Phase A	Static IPs + SSH on all 3 Pis	Laptop can SSH to all 3 Pis without display
Phase B	Chrony NTP sync	chronyc tracking shows laptop IP on all Pis
Phase C	FRRouting installed	sudo systemctl status frr shows active
Phase D	MPLS kernel modules	ip link show type mpls shows mpls0
Phase E	OSPF between Pis	show ip ospf neighbor shows FULL on all
Phase F	BGP sessions	show ip bgp summary shows Established
Phase G	MPLS LDP labels	show mpls table shows label entries
Phase H	VRFs configured	show vrf shows MISSION_CONTROL and ADMIN
Phase I	CE namespaces	ping from ce-a namespace reaches ce-b namespace
Phase J	RSVP-TE LSPs	show mpls traffic-eng tunnels shows Up
Phase K	IPSec tunnels	sudo ipsec status shows ESTABLISHED
Phase L	iPerf3 traffic	iPerf3 reports jitter and packet loss

02 Before You Begin

Hardware Checklist

Item	Status	Notes
3x Raspberry Pi 4 Model B 8GB	Have	station1, station2, station3
3x 32GB SD cards (flashed)	Have	Ubuntu 22.04.5 LTS ARM64
3x power supplies (5V 3A USB-C)	Have	Official RPi4 supply recommended
1x 5-port gigabit switch	Have	TP-Link TL-SG105 or similar
4x ethernet cables	Have	Cat5e or Cat6
1x USB-C to ethernet adapter	Need	For laptop -- Honor MagicBook has no ethernet
1x external display + keyboard	Have	For initial setup, 1 Pi at a time
Laptop with Ubuntu 22.04 dual boot	Pending	Set up before Phase F

Important: Internet Required During Setup

WARNING: All Pis need internet access during the setup phase to download packages (FRRouting, strongSwan, iPerf3, pmacct etc). Connect the switch to your home router during setup. You will disconnect it before the demo.

What Was Set in Raspberry Pi Imager

When you flashed the cards, Raspberry Pi Imager's advanced options allowed you to pre-configure settings. The guide assumes you set:

Setting	Assumed Value	If Different
Hostname	station1 / station2 / station3	Run: <code>sudo hostnamectl set-hostname stationX</code>
Username	ubuntu	Replace 'ubuntu' in all commands with your username
Password	Custom (you set it)	Use whatever you chose
SSH	Enabled	If not: <code>sudo systemctl enable --now ssh</code>

Setting	Assumed Value	If Different
WiFi	Not configured (using ethernet)	Disconnect WiFi if it auto-connected

Ethernet Interface Name

On Ubuntu 22.04 Server for Raspberry Pi 4, the ethernet interface is named eth0. Verify this on first boot by running:

```
ip link show
```

Look for a line starting with "2:" -- it will show the interface name. If it is NOT eth0 (e.g. it shows enp3s0 or eth1), replace every occurrence of 'eth0' in this guide with that name.

NOTE: Raspberry Pi 4 has one physical ethernet port. It will appear as eth0 on standard Ubuntu 22.04 ARM64 images.

03 Station 1 -- First Boot Setup

Connect display and keyboard to Station 1 only. Station 2 and 3 will be configured headlessly via SSH after Station 1 is online.

Physical Setup

- Step 1.** Insert the flashed 32GB SD card into Station 1 (Pi 1).
- Step 2.** Connect ethernet cable from Station 1 to the switch.
- Step 3.** Connect HDMI to the external display.
- Step 4.** Connect USB keyboard.
- Step 5.** Connect power supply -- Pi will boot automatically.
- Step 6.** Wait 60-90 seconds for full boot. You will see the Ubuntu login prompt.

First Login

```
Login: ubuntu
Password: [whatever you set in Imager, or 'ubuntu' if default]
```

NOTE: If you set a password in Imager, use that. If Ubuntu 22.04 was flashed with default settings, the first login is ubuntu/ubuntu and it FORCES you to change the password immediately.

Verify Hostname

```
hostname
station1
```

IF YOU SEE THIS ERROR: If hostname shows 'ubuntu' or something else: `sudo hostnamectl set-hostname station1 && exec bash`

Update the System

This is mandatory. Do not skip.

```
sudo apt update
sudo apt upgrade -y
```

This will take 3-8 minutes depending on internet speed. Wait for it to complete fully.

Install Essential Tools

```
sudo apt install -y \
  net-tools \
  curl \
  wget \
  vim \
  nano \
  htop \
  tree \
```



```
tcpdump \  
traceroute \  
iputils-ping \  
iproute2 \  
dnsutils \  
lsof \  
nload \  
chrony
```

Configure Static IP -- Station 1

Ubuntu 22.04 uses Netplan for network configuration. The default config file is in /etc/netplan/. Check what files exist:

```
ls /etc/netplan/
```

You will see a file named something like '50-cloud-init.yaml'. Open it:

```
sudo nano /etc/netplan/50-cloud-init.yaml
```

Delete ALL existing content and replace with exactly this (use spaces, NOT tabs -- YAML is space-sensitive):

```
network:  
  version: 2  
  renderer: networkd  
  ethernets:  
    eth0:  
      dhcp4: false  
      addresses:  
        - 192.168.1.10/24  
      routes:  
        - to: default  
          via: 192.168.1.1  
      nameservers:  
        addresses: [8.8.8.8, 1.1.1.1]
```

WARNING: YAML is indentation-sensitive. Every space must be exactly right. Use spaces only -- never Tab. Copy the indentation exactly as shown.

Save the file (Ctrl+O, Enter, Ctrl+X in nano). Then apply and test:

```
sudo netplan generate  
sudo netplan apply
```

Wait 3 seconds, then verify the IP was applied:

```
ip addr show eth0  
  
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 ...  
    link/ether dc:a6:32:xx:xx:xx brd ff:ff:ff:ff:ff:ff  
    inet 192.168.1.10/24 brd 192.168.1.255 scope global eth0
```

SUCCESS CHECK: You see inet 192.168.1.10/24 -- static IP is set correctly.

Configure Loopback for Router-ID

FRRouting uses the loopback interface for BGP and LDP router-IDs. Assign it now via Netplan. Edit the same file:

```
sudo nano /etc/netplan/50-cloud-init.yaml
```

Update to include the loopback:

```
network:
  version: 2
  renderer: networkd
  ethernets:
    eth0:
      dhcp4: false
      addresses:
        - 192.168.1.10/24
      routes:
        - to: default
          via: 192.168.1.1
      nameservers:
        addresses: [8.8.8.8, 1.1.1.1]
  lo:
    addresses:
      - 127.0.0.1/8
      - 10.1.1.1/32
```

```
sudo netplan apply
```

```
ip addr show lo
```

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 ...
    inet 127.0.0.1/8 scope host lo
    inet 10.1.1.1/32 scope global lo
```

Enable SSH (if not already active)

```
sudo systemctl enable ssh
sudo systemctl start ssh
sudo systemctl status ssh
```

```
ssh.service - OpenBSD Secure Shell server
Loaded: loaded (/lib/systemd/system/ssh.service)
Active: active (running)
```

Test SSH from Laptop

On your LAPTOP (not on the Pi), open a terminal and run:

```
ssh ubuntu@192.168.1.10
```

Accept the host key fingerprint (type 'yes'). You should land at the station1 prompt. Type 'exit' to return to laptop. From this point, you no longer need the display connected to Station 1.

SUCCESS CHECK: SSH works from laptop to Station 1. Disconnect the display and keyboard from Station 1.

04 Station 2 -- First Boot Setup

Station 2 is identical to Station 1 setup, but uses 192.168.1.20 and loopback 10.1.2.1. Connect the display to Station 2. Station 1 stays running connected to the switch.

Physical Setup and First Login

- Step 1.** Move display and keyboard to Station 2.
- Step 2.** Connect ethernet from Station 2 to the switch.
- Step 3.** Power on Station 2 and log in (same credentials as Station 1).

Verify Hostname

```
hostname
station2
```

Update and Install Packages

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y net-tools curl wget vim nano htop tree \
tcpdump traceroute iputils-ping iproute2 dnsutils lsof nload chrony
```

Static IP for Station 2

```
sudo nano /etc/netplan/50-cloud-init.yaml

network:
  version: 2
  renderer: networkd
  ethernets:
    eth0:
      dhcp4: false
      addresses:
        - 192.168.1.20/24
      routes:
        - to: default
          via: 192.168.1.1
      nameservers:
        addresses: [8.8.8.8, 1.1.1.1]
  lo:
    addresses:
      - 127.0.0.1/8
      - 10.1.2.1/32

sudo netplan apply
ip addr show eth0

    inet 192.168.1.20/24 brd 192.168.1.255 scope global eth0
```

Enable SSH and Test from Laptop

```
sudo systemctl enable ssh && sudo systemctl start ssh
```

From the LAPTOP:

```
ssh ubuntu@192.168.1.20  
# Also verify station1 is still reachable:  
ssh ubuntu@192.168.1.10
```

SUCCESS CHECK: Both Station 1 and Station 2 are SSH-accessible from laptop. Disconnect display from Station 2.

05 Station 3 -- First Boot Setup

Station 3 is the P core router. It uses 192.168.1.30 and loopback 10.1.3.1. This is the last Pi that needs the physical display.

Physical Setup and First Login

- Step 1.** Move display and keyboard to Station 3.
- Step 2.** Connect ethernet from Station 3 to the switch.
- Step 3.** Power on and log in.

Verify Hostname

```
hostname
station3
```

Update and Install Packages

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y net-tools curl wget vim nano htop tree \
tcpdump traceroute iputils-ping iproute2 dnsutils lsof nload chrony
```

Static IP for Station 3

```
sudo nano /etc/netplan/50-cloud-init.yaml

network:
  version: 2
  renderer: networkd
  ethernets:
    eth0:
      dhcp4: false
      addresses:
        - 192.168.1.30/24
      routes:
        - to: default
          via: 192.168.1.1
      nameservers:
        addresses: [8.8.8.8, 1.1.1.1]
  lo:
    addresses:
      - 127.0.0.1/8
      - 10.1.3.1/32

sudo netplan apply
ip addr show eth0

    inet 192.168.1.30/24 brd 192.168.1.255 scope global eth0

sudo systemctl enable ssh && sudo systemctl start ssh
```

From LAPTOP -- test all three:

```
ssh ubuntu@192.168.1.10 'echo station1 OK'
ssh ubuntu@192.168.1.20 'echo station2 OK'
ssh ubuntu@192.168.1.30 'echo station3 OK'
```

```
station1 OK
station2 OK
station3 OK
```

SUCCESS CHECK: All three stations are SSH-accessible. Display is no longer needed. All remaining work is done from the laptop terminal via SSH.

06 Laptop Preparation

The laptop runs Ubuntu 22.04 (dual boot). All remaining commands are run FROM the laptop either locally or via SSH to the Pis. Do this chapter on the laptop itself.

Set Laptop Static IP

Connect the USB-C to ethernet adapter and plug ethernet cable into the switch. Then set a static IP on the laptop. On Ubuntu 22.04:

```
ip link show
```

Find your ethernet interface name -- it may be eth0, enp3s0, or named after the USB adapter (e.g. enx...). Use that name below.

```
sudo nano /etc/netplan/01-netcfg.yaml

network:
  version: 2
  renderer: networkd
  ethernets:
    enp3s0:
      dhcp4: false
      addresses:
        - 192.168.1.1/24
      nameservers:
        addresses: [8.8.8.8, 1.1.1.1]
```

NOTE: Replace enp3s0 with your actual interface name. No default gateway needed on laptop since Pis use laptop as their gateway for internet.

```
sudo netplan apply
```

Set Up SSH Key Authentication (Optional but Recommended)

This lets you SSH to Pis without typing password every time:

```
# Generate key if you don't have one:
ssh-keygen -t ed25519 -C 'deca-lab'
# Press Enter for all prompts (no passphrase)

# Copy key to each Pi:
ssh-copy-id ubuntu@192.168.1.10
ssh-copy-id ubuntu@192.168.1.20
ssh-copy-id ubuntu@192.168.1.30
```

After this, SSH will no longer ask for a password.

Create SSH Shortcuts

```
nano ~/.ssh/config

Host s1
  HostName 192.168.1.10
```

```
User ubuntu

Host s2
  HostName 192.168.1.20
  User ubuntu

Host s3
  HostName 192.168.1.30
  User ubuntu
```

Now you can type `ssh s1` instead of `ssh ubuntu@192.168.1.10`. All remaining chapters use these shortcuts.

07 Network Verification

Confirm all 4 devices can reach each other before installing any routing software.

Ping All Stations from Laptop

```
ping -c 3 192.168.1.10
ping -c 3 192.168.1.20
ping -c 3 192.168.1.30

PING 192.168.1.10: 64 bytes from 192.168.1.10: icmp_seq=1 ttl=64 time=0.5 ms
PING 192.168.1.20: 64 bytes from 192.168.1.20: icmp_seq=1 ttl=64 time=0.6 ms
PING 192.168.1.30: 64 bytes from 192.168.1.30: icmp_seq=1 ttl=64 time=0.4 ms
```

SUCCESS CHECK: All three Pis respond. You have Layer 3 connectivity.

Ping Between Pis via SSH

```
# From station1, ping station2 and station3:
ssh s1 'ping -c 3 192.168.1.20 && ping -c 3 192.168.1.30'

# From station3, ping station1 and station2:
ssh s3 'ping -c 3 192.168.1.10 && ping -c 3 192.168.1.20'
```

SUCCESS CHECK: All Pis can reach each other. Network foundation is solid.

08 Chrony NTP Synchronisation

Log timestamps from Pis and laptop MUST be synchronised. Without this, correlating events across devices (e.g. BGP event on station1 with metric spike on station3) is impossible. The laptop acts as the NTP server for all Pis.

Step 1: Configure Laptop as NTP Server

```
sudo nano /etc/chrony/chrony.conf
```

Find and add these two lines (add them at the top, before any pool lines):

```
local stratum 1
allow 192.168.1.0/24

sudo systemctl restart chrony
sudo systemctl enable chrony
chronyc sources
```

```
MS Name/IP address         Stratum Poll Reach LastRx Last sample
=====
^* LOCAL(0)                0      4   377    11    +0ns[ +0ns] +/-  0ns
```

Step 2: Configure All 3 Pis as NTP Clients

Run this on each Pi. Using a for loop from laptop:

```
for HOST in s1 s2 s3; do
  echo "--- Configuring $HOST ---"
  ssh $HOST 'sudo bash -c \''
    cp /etc/chrony/chrony.conf /etc/chrony/chrony.conf.bak
    grep -v "^pool" /etc/chrony/chrony.conf.bak | \
    grep -v "^server" > /etc/chrony/chrony.conf
    echo "server 192.168.1.1 iburst" >> /etc/chrony/chrony.conf
    systemctl restart chrony
  \''
done
```

NOTE: If the for loop syntax causes issues, SSH into each Pi individually and edit /etc/chrony/chrony.conf manually: remove all pool/server lines and add: server 192.168.1.1 iburst

Step 3: Verify Sync on All Pis

Wait 30 seconds after restarting chrony, then check:

```
for HOST in s1 s2 s3; do
  echo "=== $HOST ==="
  ssh $HOST 'chronyc tracking | grep -E "Reference|offset"'
done

=== s1 ===
Reference ID      : C0A80101 (192.168.1.1)
System time       : 0.000002341 seconds fast of NTP time
=== s2 ===
Reference ID      : C0A80101 (192.168.1.1)
```

```
System time      : 0.000001872 seconds fast of NTP time  
=== s3 ===  
Reference ID     : C0A80101 (192.168.1.1)  
System time      : 0.000003102 seconds fast of NTP time
```

SUCCESS CHECK: Reference ID shows 192.168.1.1 (laptop). All Pis sync to laptop. System time offset is under 1ms. NTP is working.

09 System-Wide Package Preparation

Install the kernel modules and packages needed for MPLS and routing on all three stations. Run these commands on each Pi via SSH.

Enable IP Forwarding on All Pis

Without this, Pis will not forward packets -- they will only route to themselves. This is the most critical sysctl for a router.

```
for HOST in s1 s2 s3; do
    ssh $HOST 'sudo bash -c \''
        echo "net.ipv4.ip_forward = 1" >> /etc/sysctl.conf
        sysctl -p
    \''
done

# Verify on any Pi:
ssh s1 'sysctl net.ipv4.ip_forward'

net.ipv4.ip_forward = 1
```

Enable MPLS Kernel Modules

MPLS label switching requires two kernel modules. Load them and make them persistent.

```
for HOST in s1 s2 s3; do
    echo "--- $HOST ---"
    ssh $HOST 'sudo bash -c \''
        modprobe mpls_router
        modprobe mpls_ip_tunnel
        echo mpls_router >> /etc/modules
        echo mpls_ip_tunnel >> /etc/modules
    \''
done
```

Configure MPLS sysctl Parameters

These settings enable MPLS label processing on the eth0 and loopback interfaces.

```
for HOST in s1 s2 s3; do
    ssh $HOST 'sudo bash -c \''
        cat >> /etc/sysctl.conf << EOF
net.mpls.conf.eth0.input = 1
net.mpls.conf.lo.input = 1
net.mpls.platform_labels = 1048575
EOF
        sysctl -p
    \''
done

# Verify MPLS is active on eth0:
ssh s1 'sysctl net.mpls.conf.eth0.input'
```

```
net.mpls.conf.eth0.input = 1
```

SUCCESS CHECK: MPLS kernel modules loaded. Label forwarding enabled on eth0 and lo.

10 FRRouting Installation

FRRouting (FRR) is the open-source routing suite that implements OSPF, BGP, LDP, and RSVP-TE. Install it on all three stations.

Add FRR Repository and Install

Run on each Pi. This requires internet access.

```
for HOST in s1 s2 s3; do
    echo "=== Installing FRR on $HOST ==="
    ssh $HOST 'sudo bash -c \''
        apt install -y curl gnupg lsb-release
        curl -s https://deb.frrouting.org/frr/keys.gpg \
            | tee /usr/share/keyrings/frrouting.gpg > /dev/null
        echo "deb [signed-by=/usr/share/keyrings/frrouting.gpg] \
            https://deb.frrouting.org/frr \
            $(lsb_release -s -c) frr-stable" \
            | tee /etc/apt/sources.list.d/frr.list
        apt update
        apt install -y frr frr-pythontools
    \''
done
```

NOTE: Installation takes 2-5 minutes per Pi. The FRR package installs zebra (core), ospfd, bgpd, ldpd and other daemons. Only some are enabled by default.

Enable Required FRR Daemons

FRR only runs daemons you explicitly enable. Edit /etc/frr/daemons on ALL three stations to enable: zebra, ospfd, bgpd, ldpd.

```
for HOST in s1 s2 s3; do
    ssh $HOST 'sudo bash -c \''
        sed -i "s/^zebra=no/zebra=yes/" /etc/frr/daemons
        sed -i "s/^ospfd=no/ospfd=yes/" /etc/frr/daemons
        sed -i "s/^bgpd=no/bgpd=yes/" /etc/frr/daemons
        sed -i "s/^ldpd=no/ldpd=yes/" /etc/frr/daemons
    \''
done
```

Start FRR on All Stations

```
for HOST in s1 s2 s3; do
    ssh $HOST 'sudo systemctl enable frr && sudo systemctl start frr'
done

# Check status on each:
for HOST in s1 s2 s3; do
    echo "=== $HOST ==="
    ssh $HOST 'sudo systemctl status frr --no-pager | head -5'
done
```

```
=== s1 ===
frr.service - FRRouting
  Active: active (running)
=== s2 ===
  Active: active (running)
=== s3 ===
  Active: active (running)
```

SUCCESS CHECK: FRR is running on all three stations with zebra, ospfd, bgpd, and ldpd enabled.

Access vtysh -- The FRR CLI

vttysh is FRRouting's unified command-line interface. All routing configuration is done here.

```
# SSH into station1 and open vtysh:
ssh s1
sudo vtysh

Hello, this is FRRouting (version 9.x).
Copyright 1996-2005 Kunihiro Ishiguro, et al.

station1#
```

You are now inside the FRR CLI. Type 'exit' to leave. Configuration mode is entered with 'configure terminal' (or 'conf t').

```
# Useful vtysh commands:
show version           # FRR version info
show interface         # Interface status
show running-config    # Current config
write memory           # Save config to disk
exit                   # Leave vtysh
```

11 OSPF Configuration

OSPF is the IGP (Interior Gateway Protocol) that discovers neighbors and builds the routing table inside the MPLS domain. All three Pis must form OSPF adjacencies with each other.

Station 1 -- OSPF Configuration

```
ssh s1
sudo vtvsh
configure terminal
!
hostname PE-1
!
interface lo
 ip address 10.1.1.1/32
!
interface eth0
 ip address 192.168.1.10/24
 ip ospf cost 10
!
router ospf
 ospf router-id 10.1.1.1
 passive-interface default
 no passive-interface eth0
 network 192.168.1.0/24 area 0
 network 10.1.1.1/32 area 0
 timers throttle spf 100 1000 10000
!
end
write memory
```

NOTE: 'passive-interface default' suppresses OSPF hellos on all interfaces. 'no passive-interface eth0' re-enables them on eth0 only. This is correct -- OSPF should only peer on eth0.

Station 2 -- OSPF Configuration

```
ssh s2
sudo vtvsh
configure terminal
hostname PE-2
!
interface lo
 ip address 10.1.2.1/32
!
interface eth0
 ip address 192.168.1.20/24
 ip ospf cost 10
!
router ospf
 ospf router-id 10.1.2.1
```



```

passive-interface default
no passive-interface eth0
network 192.168.1.0/24 area 0
network 10.1.2.1/32 area 0
timers throttle spf 100 1000 10000
!
end
write memory

```

Station 3 -- OSPF Configuration

```

ssh s3
sudo vtysh

configure terminal
hostname P-CORE
!
interface lo
ip address 10.1.3.1/32
!
interface eth0
ip address 192.168.1.30/24
ip ospf cost 10
!
router ospf
ospf router-id 10.1.3.1
passive-interface default
no passive-interface eth0
network 192.168.1.0/24 area 0
network 10.1.3.1/32 area 0
timers throttle spf 100 1000 10000
!
end
write memory

```

Verify OSPF Adjacencies

Wait 30 seconds for OSPF to converge, then check from any Pi:

```
ssh s1 'sudo vtysh -c "show ip ospf neighbor"'
```

Neighbor ID	Pri	State	Dead Time	Address	Interface
10.1.2.1	1	Full/DR	00:00:38	192.168.1.20	eth0:192.168.1.10
10.1.3.1	1	Full/Backup	00:00:38	192.168.1.30	eth0:192.168.1.10

SUCCESS CHECK: State shows 'Full' for both neighbors. OSPF adjacency is established. All three Pis see each other.

IF YOU SEE THIS ERROR: If State shows 'Init' or 'ExStart' after 60 seconds: check that no firewall is blocking UDP port 89 (sudo ufw status), and that OSPF is enabled on eth0 (no passive-interface eth0 is in the config).

Verify Routes Learned

```
ssh s1 'sudo vtysh -c "show ip route ospf"'  
O    10.1.2.1/32 [110/20] via 192.168.1.20, eth0, 00:01:14  
O    10.1.3.1/32 [110/20] via 192.168.1.30, eth0, 00:01:14
```

SUCCESS CHECK: Station 1 has learned the loopback routes of Station 2 (10.1.2.1) and Station 3 (10.1.3.1) via OSPF. Routing is working.

Test Loopback Reachability

```
# From laptop, ping all loopbacks via station1:  
ssh s1 'ping -c 3 10.1.2.1 && ping -c 3 10.1.3.1'  
  
PING 10.1.2.1: 64 bytes from 10.1.2.1: icmp_seq=1 ttl=64 time=0.8 ms  
PING 10.1.3.1: 64 bytes from 10.1.3.1: icmp_seq=1 ttl=64 time=0.7 ms
```

12 BGP Configuration

BGP carries VPN routing information (VPNv4) between PE routers. All three routers run iBGP (same AS 65001). Station1 (PE-1) and Station2 (PE-2) exchange VPNv4 routes for L3VPN. Station3 (P-CORE) acts as a BGP route reflector.

Station 3 -- BGP Route Reflector

Configure Station 3 as the route reflector. PE-1 and PE-2 peer to it, and it reflects routes between them.

```
ssh s3 'sudo vtvsh'

configure terminal
!
router bgp 65001
  bgp router-id 10.1.3.1
  no bgp ebgp-requires-policy
  bgp log-neighbor-changes
  !
  neighbor 10.1.1.1 remote-as 65001
  neighbor 10.1.1.1 update-source lo
  neighbor 10.1.1.1 route-reflector-client
  !
  neighbor 10.1.2.1 remote-as 65001
  neighbor 10.1.2.1 update-source lo
  neighbor 10.1.2.1 route-reflector-client
  !
  address-family ipv4 unicast
    neighbor 10.1.1.1 activate
    neighbor 10.1.1.1 route-reflector-client
    neighbor 10.1.2.1 activate
    neighbor 10.1.2.1 route-reflector-client
    neighbor 10.1.1.1 next-hop-self
    neighbor 10.1.2.1 next-hop-self
  exit-address-family
  !
  address-family ipv4 vpn
    neighbor 10.1.1.1 activate
    neighbor 10.1.1.1 route-reflector-client
    neighbor 10.1.2.1 activate
    neighbor 10.1.2.1 route-reflector-client
  exit-address-family
!
end
write memory
```

Station 1 -- BGP PE-1 Configuration

```
ssh s1 'sudo vtvsh'

configure terminal
!
```

```

router bgp 65001
  bgp router-id 10.1.1.1
  no bgp ebgp-requires-policy
  bgp log-neighbor-changes
  !
  neighbor 10.1.3.1 remote-as 65001
  neighbor 10.1.3.1 update-source lo
  !
  address-family ipv4 unicast
    neighbor 10.1.3.1 activate
    neighbor 10.1.3.1 next-hop-self
  exit-address-family
  !
  address-family ipv4 vpn
    neighbor 10.1.3.1 activate
  exit-address-family
  !
end
write memory

```

Station 2 -- BGP PE-2 Configuration

```

ssh s2 'sudo vtvsh'

configure terminal
!
router bgp 65001
  bgp router-id 10.1.2.1
  no bgp ebgp-requires-policy
  bgp log-neighbor-changes
  !
  neighbor 10.1.3.1 remote-as 65001
  neighbor 10.1.3.1 update-source lo
  !
  address-family ipv4 unicast
    neighbor 10.1.3.1 activate
    neighbor 10.1.3.1 next-hop-self
  exit-address-family
  !
  address-family ipv4 vpn
    neighbor 10.1.3.1 activate
  exit-address-family
  !
end
write memory

```

Verify BGP Sessions

```
ssh s1 'sudo vtysh -c "show ip bgp summary"'
```

```
BGP router identifier 10.1.1.1, local AS number 65001
```

Neighbor	V	AS	MsgRcvd	MsgSent	TblVer	InQ	OutQ	Up/Down	State/PfxRcd
10.1.3.1	4	65001	45	42	1	0	0	00:03:12	3

SUCCESS CHECK: BGP session with Station 3 (route reflector) is Established. State/PfxRcd shows a number (not 'Active' or 'Connect').

IF YOU SEE THIS ERROR: If State shows 'Active' after 2 minutes: BGP cannot reach the neighbor. Check OSPF has learned the loopback routes (ping 10.1.3.1 from station1). Also check 'update-source lo' is set -- BGP must source sessions from the loopback IP.

13 MPLS LDP Configuration

LDP (Label Distribution Protocol) assigns and distributes MPLS labels between routers. After LDP is running, packets between Pis will be label-switched instead of IP-routed through the core.

Station 1 -- LDP Configuration

```
ssh s1 'sudo vtysh'
configure terminal
!
mpls ldp
router-id 10.1.1.1
!
address-family ipv4
discovery transport-address 10.1.1.1
!
interface eth0
!
interface lo
!
exit-address-family
!
end
write memory
```

Station 2 -- LDP Configuration

```
ssh s2 'sudo vtysh'
configure terminal
!
mpls ldp
router-id 10.1.2.1
!
address-family ipv4
discovery transport-address 10.1.2.1
!
interface eth0
!
interface lo
!
exit-address-family
!
end
write memory
```

Station 3 -- LDP Configuration

```
ssh s3 'sudo vtysh'
```

```

configure terminal
!
mpls ldp
router-id 10.1.3.1
!
address-family ipv4
discovery transport-address 10.1.3.1
!
interface eth0
!
interface lo
!
exit-address-family
!
end
write memory

```

Verify LDP Discovery and Sessions

```
ssh s1 'sudo vtysh -c "show mpls ldp discovery"'
```

AF	ID	Type	Source	Holdtime	Interval
ipv4	10.1.2.1:0	Link	eth0	15	5
ipv4	10.1.3.1:0	Link	eth0	15	5

```
ssh s1 'sudo vtysh -c "show mpls ldp neighbor"'
```

AF	ID	State	Up/Down	Holdtime	KeepaliveTimer	PeerHoldtime
ipv4	10.1.2.1	OPERATIONAL	00:02:14	45	15	45
ipv4	10.1.3.1	OPERATIONAL	00:02:11	45	15	45

SUCCESS CHECK: LDP neighbors show OPERATIONAL. Labels are being distributed.

Verify MPLS Label Table

```
ssh s1 'sudo vtysh -c "show mpls table"'
```

Inbound Label	Type	Nexthop	Outbound Label
16	LDP	192.168.1.30	implicit-null
17	LDP	192.168.1.20	implicit-null
18	LDP	192.168.1.30	16

SUCCESS CHECK: MPLS label table has entries. Packets destined for loopback IPs will now be label-switched through the network.

IF YOU SEE THIS ERROR: If the label table is empty after 60 seconds: ensure MPLS is enabled on eth0 (sysctl net.mpls.conf.eth0.input = 1) and that OSPF routes exist for the loopback IPs (show ip route ospf).

14 VRF Configuration for L3VPN

VRFs create logically isolated routing tables on the PE routers. Station 1 and Station 2 each host two VRFs: MISSION_CONTROL and ADMIN. This simulates different departments or customers sharing the same physical MPLS network.

Create VRFs at the OS Level -- Station 1

```
ssh s1

# Create VRF kernel objects
sudo ip link add vrf-mission type vrf table 100
sudo ip link add vrf-admin type vrf table 200
sudo ip link set dev vrf-mission up
sudo ip link set dev vrf-admin up

# Verify VRFs exist:
ip vrf show
```

Name	Table

vrf-admin	200
vrf-mission	100

Make VRFs Persistent -- Station 1

VRFs created with 'ip link' are lost on reboot. Use systemd-networkd .netdev files to persist them.

```
sudo nano /etc/systemd/network/10-vrf-mission.netdev

[NetDev]
Name=vrf-mission
Kind=vrf

[VRF]
Table=100

sudo nano /etc/systemd/network/11-vrf-admin.netdev

[NetDev]
Name=vrf-admin
Kind=vrf

[VRF]
Table=200

sudo systemctl restart systemd-networkd
ip vrf show
```

Repeat VRF Setup on Station 2

```
ssh s2

sudo ip link add vrf-mission type vrf table 100
sudo ip link add vrf-admin type vrf table 200
sudo ip link set dev vrf-mission up
```



```
sudo ip link set dev vrf-admin up
```

Also create the same .netdev files on Station 2.

Configure VPN BGP on Station 1

```
ssh s1 'sudo vtysh'
configure terminal
!
vrf vrf-mission
vni 100
exit-vrf
!
vrf vrf-admin
vni 200
exit-vrf
!
router bgp 65001
!
address-family ipv4 vpn
neighbor 10.1.3.1 activate
exit-address-family
!
router bgp 65001 vrf vrf-mission
bgp router-id 10.1.1.1
!
address-family ipv4 unicast
label vpn export auto
rd vpn export 65001:100
rt vpn both 65001:100
export vpn
import vpn
exit-address-family
!
router bgp 65001 vrf vrf-admin
bgp router-id 10.1.1.1
!
address-family ipv4 unicast
label vpn export auto
rd vpn export 65001:200
rt vpn both 65001:200
export vpn
import vpn
exit-address-family
!
end
write memory
```

Configure VPN BGP on Station 2

```
ssh s2 'sudo vtysh'
```

```

configure terminal
!
vrf vrf-mission
  vni 100
exit-vrf
!
vrf vrf-admin
  vni 200
exit-vrf
!
router bgp 65001
!
  address-family ipv4 vpn
    neighbor 10.1.3.1 activate
  exit-address-family
!
router bgp 65001 vrf vrf-mission
  bgp router-id 10.1.2.1
!
  address-family ipv4 unicast
    label vpn export auto
    rd vpn export 65001:100
    rt vpn both 65001:100
    export vpn
    import vpn
  exit-address-family
!
router bgp 65001 vrf vrf-admin
  bgp router-id 10.1.2.1
!
  address-family ipv4 unicast
    label vpn export auto
    rd vpn export 65001:200
    rt vpn both 65001:200
    export vpn
    import vpn
  exit-address-family
!
end
write memory

```

Verify VRFs and VPN Routes

```

ssh s1 'sudo vtysh -c "show bgp vrf vrf-mission ipv4 unicast"'
ssh s1 'sudo vtysh -c "show ip bgp vpnv4 all"'

```

SUCCESS CHECK: VRF configuration is complete. MISSION_CONTROL and ADMIN VRFs are isolated routing tables on each PE.

15 Network Namespace CE Simulation

CE (Customer Edge) routers are simulated using Linux network namespaces. Station 1 hosts the CE-A namespace (Bangalore site) and Station 2 hosts the CE-B namespace (Sriharikota site). Traffic from CE-A reaches CE-B through the MPLS L3VPN.

Station 1 -- Create CE-A Namespace

```
ssh s1

# Create the CE-A namespace
sudo ip netns add ce-a

# Create a veth pair:
# veth-pe-cea: lives in PE-1 (main namespace) inside vrf-mission
# veth-cea-pe: lives in ce-a namespace
sudo ip link add veth-pe-cea type veth peer name veth-cea-pe

# Move one end into CE-A namespace
sudo ip link set veth-cea-pe netns ce-a

# Assign PE-1 side into vrf-mission and give it an IP
sudo ip link set veth-pe-cea master vrf-mission
sudo ip addr add 10.10.1.2/30 dev veth-pe-cea
sudo ip link set veth-pe-cea up

# Configure CE-A namespace interface
sudo ip netns exec ce-a ip addr add 10.10.1.1/30 dev veth-cea-pe
sudo ip netns exec ce-a ip link set veth-cea-pe up
sudo ip netns exec ce-a ip link set lo up
sudo ip netns exec ce-a ip addr add 10.100.1.1/32 dev lo
sudo ip netns exec ce-a ip route add default via 10.10.1.2
```

Station 1 -- Advertise CE-A Prefix into MPLS VPN

```
sudo vtysh

configure terminal
!
router bgp 65001 vrf vrf-mission
 address-family ipv4 unicast
  network 10.10.1.0/30
  network 10.100.1.1/32
 exit-address-family
!
end
write memory
```

Station 2 -- Create CE-B Namespace

```
ssh s2
```

```

sudo ip netns add ce-b

sudo ip link add veth-pe-ceb type veth peer name veth-ceb-pe
sudo ip link set veth-ceb-pe netns ce-b

sudo ip link set veth-pe-ceb master vrf-mission
sudo ip addr add 10.10.2.2/30 dev veth-pe-ceb
sudo ip link set veth-pe-ceb up

sudo ip netns exec ce-b ip addr add 10.10.2.1/30 dev veth-ceb-pe
sudo ip netns exec ce-b ip link set veth-ceb-pe up
sudo ip netns exec ce-b ip link set lo up
sudo ip netns exec ce-b ip addr add 10.100.2.1/32 dev lo
sudo ip netns exec ce-b ip route add default via 10.10.2.2

sudo vtysh

configure terminal
!
router bgp 65001 vrf vrf-mission
 address-family ipv4 unicast
   network 10.10.2.0/30
   network 10.100.2.1/32
 exit-address-family
!
end
write memory

```

Make Namespaces Persistent

Network namespaces are lost on reboot. Create a startup script to recreate them:

```
sudo nano /etc/rc.local
```

On Station 1, add (before the 'exit 0' line):

```

#!/bin/bash
ip netns add ce-a 2>/dev/null || true
ip link add veth-pe-cea type veth peer name veth-cea-pe 2>/dev/null || true
ip link set veth-cea-pe netns ce-a 2>/dev/null || true
ip link set veth-pe-cea master vrf-mission 2>/dev/null || true
ip addr add 10.10.1.2/30 dev veth-pe-cea 2>/dev/null || true
ip link set veth-pe-cea up
ip netns exec ce-a ip addr add 10.10.1.1/30 dev veth-cea-pe 2>/dev/null || true
ip netns exec ce-a ip link set veth-cea-pe up
ip netns exec ce-a ip link set lo up
ip netns exec ce-a ip addr add 10.100.1.1/32 dev lo 2>/dev/null || true
ip netns exec ce-a ip route add default via 10.10.1.2 2>/dev/null || true
exit 0

sudo chmod +x /etc/rc.local

```

Verify End-to-End CE-A to CE-B Connectivity

```

# From CE-A namespace on station1, ping CE-B loopback on station2:
ssh s1 'sudo ip netns exec ce-a ping -c 3 10.100.2.1'

```

```
PING 10.100.2.1 (10.100.2.1) 56(84) bytes of data.  
64 bytes from 10.100.2.1: icmp_seq=1 ttl=62 time=1.2 ms  
64 bytes from 10.100.2.1: icmp_seq=2 ttl=62 time=0.9 ms  
64 bytes from 10.100.2.1: icmp_seq=3 ttl=62 time=1.1 ms
```

SUCCESS CHECK: CE-A can reach CE-B through the MPLS L3VPN. Traffic from 10.100.1.1 (Bangalore) reaches 10.100.2.1 (Sriharikota) via the label-switched path through Station 3 (P core).

16 MPLS-TE with RSVP-TE

MPLS Traffic Engineering (MPLS-TE) allows explicit control of which path traffic takes, with bandwidth reservations. RSVP-TE is the signalling protocol. This is critical for ISRO's mission-critical telemetry flows.

Enable RSVP-TE Daemon in FRR

```
for HOST in s1 s2 s3; do
    ssh $HOST 'sudo bash -c "sed -i s/^pathd=no/pathd=yes/ /etc/frr/daemons && systemctl r
    estart frr"'
done
```

NOTE: pathd is the FRR daemon for Segment Routing and Traffic Engineering. In some FRR versions, RSVP-TE is handled via ospfd directly with MPLS-TE extensions.

Enable MPLS-TE in OSPF -- All Stations

```
ssh s1 'sudo vtysh'
configure terminal
!
interface eth0
    mpls-te enable
    mpls-te link max-bw 1000000000
    mpls-te link max-rsv-bw 800000000
    mpls-te link unrsv-bw 0 1000000000
    mpls-te link unrsv-bw 1 900000000
    mpls-te link unrsv-bw 2 800000000
    mpls-te link unrsv-bw 3 700000000
    mpls-te link unrsv-bw 4 600000000
    mpls-te link unrsv-bw 5 500000000
    mpls-te link unrsv-bw 6 400000000
    mpls-te link unrsv-bw 7 300000000
!
router ospf
    mpls-te on
    mpls-te router-address 10.1.1.1
!
end
write memory
```

Repeat the same MPLS-TE configuration on Station 2 (router-address 10.1.2.1) and Station 3 (router-address 10.1.3.1).

Verify MPLS-TE

```
ssh s1 'sudo vtysh -c "show mpls-te interface"'
-- MPLS-TE router parameters --
Router-Address: 10.1.1.1 (config)
Link: eth0 (enabled)
Maximum Bandwidth: 1.000 Gbps
```

```
Maximum Reservable Bandwidth: 0.800 Gbps
```

```
ssh s1 'sudo vtysh -c "show ip ospf database opaque-area"'
```

SUCCESS CHECK: OSPF TE-LSAs are visible in the database. Traffic Engineering topology is advertised.

17 strongSwan IPSec Tunnels

IPSec tunnels simulate the SD-WAN encrypted overlay that runs on top of the MPLS underlay. Station 1 and Station 2 establish an IKEv2 IPSec tunnel. Rekey events from this tunnel are a key telemetry signal for DECA.

Install strongSwan

```
for HOST in s1 s2; do
    ssh $HOST 'sudo apt install -y strongswan strongswan-pki libcharon-extra-plugins libstrongswan-extra-plugins'
done
```

Generate PKI Keys and Certificates

Run on the LAPTOP (not on Pis). Generate a CA and certificates for both Pis.

```
mkdir -p ~/deca-pki/{ca,certs,keys,reqs}
cd ~/deca-pki

# Generate CA private key
ipsec pki --gen --type rsa --size 4096 --outform pem > ca/ca-key.pem

# Generate CA certificate
ipsec pki --self --ca --lifetime 3650 \
    --in ca/ca-key.pem --type rsa \
    --dn 'CN=DECA-CA' --outform pem > ca/ca-cert.pem

# Generate Station 1 key and cert
ipsec pki --gen --type rsa --size 2048 --outform pem > keys/station1-key.pem
ipsec pki --pub --in keys/station1-key.pem --type rsa \
    | ipsec pki --issue --lifetime 1825 \
    --cacert ca/ca-cert.pem --cakey ca/ca-key.pem \
    --dn 'CN=station1' --san '192.168.1.10' \
    --flag serverAuth --flag ikeIntermediate \
    --outform pem > certs/station1-cert.pem

# Generate Station 2 key and cert
ipsec pki --gen --type rsa --size 2048 --outform pem > keys/station2-key.pem
ipsec pki --pub --in keys/station2-key.pem --type rsa \
    | ipsec pki --issue --lifetime 1825 \
    --cacert ca/ca-cert.pem --cakey ca/ca-key.pem \
    --dn 'CN=station2' --san '192.168.1.20' \
    --flag serverAuth --flag ikeIntermediate \
    --outform pem > certs/station2-cert.pem
```

Deploy Certificates to Pis

```
cd ~/deca-pki

# Station 1:
scp ca/ca-cert.pem s1:/tmp/
```



```

scp certs/station1-cert.pem s1:/tmp/
scp keys/station1-key.pem s1:/tmp/
ssh s1 'sudo cp /tmp/ca-cert.pem /etc/ipsec.d/cacerts/
        sudo cp /tmp/station1-cert.pem /etc/ipsec.d/certs/
        sudo cp /tmp/station1-key.pem /etc/ipsec.d/private/
        sudo chmod 600 /etc/ipsec.d/private/station1-key.pem'

# Station 2:
scp ca/ca-cert.pem s2:/tmp/
scp certs/station2-cert.pem s2:/tmp/
scp keys/station2-key.pem s2:/tmp/
ssh s2 'sudo cp /tmp/ca-cert.pem /etc/ipsec.d/cacerts/
        sudo cp /tmp/station2-cert.pem /etc/ipsec.d/certs/
        sudo cp /tmp/station2-key.pem /etc/ipsec.d/private/
        sudo chmod 600 /etc/ipsec.d/private/station2-key.pem'

```

Configure IPSec on Station 1

```

ssh s1 'sudo nano /etc/ipsec.conf'

config setup
    charondebug="ike 2, knl 1, cfg 1, net 1, esp 1, dmnl 1"
    uniqueids=no

conn %default
    ikelifetime=60m
    keylife=20m
    rekeymargin=3m
    keyingtries=1
    keyexchange=ikev2
    authby=rsasig

conn deca-tunnel-s1-s2
    left=192.168.1.10
    leftid=CN=station1
    leftcert=station1-cert.pem
    leftsubnet=10.10.1.0/30
    right=192.168.1.20
    rightid=CN=station2
    rightsubnet=10.10.2.0/30
    ike=aes256-sha256-modp2048!
    esp=aes256-sha256!
    auto=start
    dpdaction=restart
    dpddelay=30s
    dpdtimeout=120s

```

Configure IPSec on Station 2

```

ssh s2 'sudo nano /etc/ipsec.conf'

config setup
    charondebug="ike 2, knl 1, cfg 1, net 1, esp 1, dmnl 1"
    uniqueids=no

```

```

conn %default
    ikelifetime=60m
    keylife=20m
    rekeymargin=3m
    keyingtries=1
    keyexchange=ikev2
    authby=rsasig

conn deca-tunnel-s1-s2
    left=192.168.1.20
    leftid=CN=station2
    leftcert=station2-cert.pem
    leftsubnet=10.10.2.0/30
    right=192.168.1.10
    rightid=CN=station1
    rightsubnet=10.10.1.0/30
    ike=aes256-sha256-modp2048!
    esp=aes256-sha256!
    auto=start
    dpdaction=restart
    dpddelay=30s
    dpdtimeout=120s

```

Start IPsec and Verifv

```

for HOST in s1 s2; do
    ssh $HOST 'sudo systemctl enable strongswan-starter && sudo ipsec restart'
done

# Wait 10 seconds then check:
sleep 10
ssh s1 'sudo ipsec status'

Security Associations (1 up, 0 connecting):
deca-tunnel-s1-s2[1]: ESTABLISHED 0 seconds ago, 192.168.1.10[CN=station1]...192.168.1.20[CN=station2]
deca-tunnel-s1-s2{1}: INSTALLED, TUNNEL, reqid 1, ESP SPIs: ...
deca-tunnel-s1-s2{1}: 10.10.1.0/30 === 10.10.2.0/30

```

SUCCESS CHECK: IPsec tunnel is ESTABLISHED. IKEv2 session active between Station 1 and Station 2. Rekey will occur automatically every 20 minutes -- visible in /var/log/charon.log.

Monitor Rekey Events in charon.log

```
ssh s1 'sudo tail -f /var/log/charon.log | grep -E "rekey|ESTABLISHED|INSTALLED"'
```

You will see rekey events appearing in the log periodically. These rekey timestamps are Channel 6 in DECA's telemetry pipeline.

18 iPerf3 Traffic Generation

iPerf3 generates realistic application traffic between CE-A and CE-B through the MPLS VPN and IPsec tunnel. It also measures jitter (Channel 5 in DECA) -- the key precursor signal for tunnel degradation.

Install iPerf3 on All Stations

```
for HOST in s1 s2 s3; do
  ssh $HOST 'sudo apt install -y iperf3'
done
```

Start iPerf3 Server on Station 2

Station 2 runs the iPerf3 server inside its CE-B namespace:

```
ssh s2 'sudo ip netns exec ce-b iperf3 -s -D --logfile /tmp/iperf3-server.log'
# -D runs as daemon, --logfile sends output to file
```

Run Traffic Test from Station 1

Station 1 sends UDP traffic from CE-A namespace to CE-B:

```
# UDP test with jitter measurement (30 seconds, 10 Mbps):
ssh s1 'sudo ip netns exec ce-a \
  iperf3 -c 10.100.2.1 -u -b 10M -t 30 -i 5'

Connecting to host 10.100.2.1, port 5201
[ ID] Interval      Transfer    Bandwidth  Jitter    Lost/Total
[  5] 0.00-5.00 sec  6.24 MBytes 10.5 Mbps  0.122 ms  0/ 4480
[  5] 5.00-10.00 sec 6.24 MBytes 10.5 Mbps  0.134 ms  0/ 4480
[  5] 10.00-15.00 sec 6.24 MBytes 10.5 Mbps  0.118 ms  0/ 4480
```

SUCCESS CHECK: iPerf3 reports jitter (~0.1-0.2ms in normal conditions) and packet loss. This is the baseline. When you inject faults, jitter will rise and packet loss will appear.

Continuous Traffic for Training Data

For ML training, you need continuous traffic running so telemetry has meaningful data to capture. Create a background traffic script:

```
nano ~/run_traffic.sh

#!/bin/bash
while true; do
  ssh s1 'sudo ip netns exec ce-a iperf3 -c 10.100.2.1 -u -b 10M -t 60 -i 10'
  sleep 5
done

chmod +x ~/run_traffic.sh && nohup ./run_traffic.sh &
```

NOTE: Run this on the laptop. It continuously sends traffic every 60-second session with 5-second gaps. Leave it running during telemetry collection and fault injection.

19 Full Simulation Verification

Run all verification checks. Every item in this checklist must pass before you consider the simulation complete and start the telemetry pipeline.

[Network Layer] All 3 Pis reachable from laptop

```
ping -c 3 192.168.1.10 && ping -c 3 192.168.1.20 && ping -c 3 192.168.1.30  
Expected: 3 pings successful for each IP
```

[NTP] All Pis synced to laptop

```
for H in s1 s2 s3; do ssh $H 'chronyc tracking | grep Reference'; done  
Expected: All show C0A80101 (192.168.1.1)
```

[OSPF] Full adjacency between all 3 Pis

```
ssh s1 'sudo vtysh -c "show ip ospf neighbor"  
Expected: Both neighbors show 'Full' state
```

[Routing] Loopbacks reachable

```
ssh s1 'ping -c 3 10.1.2.1 && ping -c 3 10.1.3.1'  
Expected: All pings successful (OSPF-learned routes)
```

[BGP] Sessions established

```
ssh s1 'sudo vtysh -c "show ip bgp summary"  
Expected: State shows a number (not Active/Connect)
```

[MPLS LDP] Labels distributed

```
ssh s1 'sudo vtysh -c "show mpls table"  
Expected: Table has at least 4 label entries
```

[VRF] Both VRFs exist

```
ssh s1 'ip vrf show'  
Expected: Shows vrf-mission (table 100) and vrf-admin (table 200)
```

[CE namespaces] CE-A and CE-B exist

```
ssh s1 'ip netns list' && ssh s2 'ip netns list'  
Expected: ce-a on station1, ce-b on station2
```

[End-to-end VPN] CE-A pings CE-B loopback

```
ssh s1 'sudo ip netns exec ce-a ping -c 3 10.100.2.1'  
Expected: 3 successful pings through MPLS L3VPN
```

[IPSec] Tunnel established

```
ssh s1 'sudo ipsec status'
```

Expected: Shows ESTABLISHED between 192.168.1.10 and 192.168.1.20

[iPerf3] Traffic flows with jitter data

```
ssh s1 'sudo ip netns exec ce-a iperf3 -c 10.100.2.1 -u -b 5M -t 10'
```

Expected: Reports Jitter in ms and Lost/Total

[FRR config saved] Config survives reboot

```
ssh s1 'sudo vtysh -c "show running-config" | head -20'
```

Expected: Hostname PE-1, interfaces, OSPF, BGP all present

SUCCESS CHECK: All 12 checks pass. The DECA network simulation is fully operational. You now have: MPLS label switching between 3 Pis, iBGP with route reflection, L3VPN with two VRFs, CE sites simulated via namespaces, IPSec tunnel with rekey events, and iPerf3 traffic generating jitter telemetry. The telemetry pipeline (Chapter 9 of the build guide) can now begin.

A Troubleshooting Reference

FRRouting Issues

Symptom	Likely Cause	Fix
<code>sudo vtysh</code> gives 'no daemons running'	FRR not started	<code>sudo systemctl start frr</code>
OSPF neighbors stuck in 'Init' state	Firewall blocking OSPF (protocol 89)	<code>sudo ufw disable</code> (for testing)
BGP state shows 'Active' forever	Loopback route not in OSPF	Check 'network 10.1.x.1/32 area 0' in ospf config
'show mpls table' is empty	MPLS not enabled on eth0	<code>sysctl net.mpls.conf.eth0.input=1</code>
LDP neighbors not appearing	LDP transport address unreachable	Ensure loopback IPs reachable: ping 10.1.x.1
VRF not created after reboot	Netdev files not created	Check <code>/etc/systemd/network/10-vrf-mission.netdev</code> exists

Network Namespace Issues

Symptom	Cause	Fix
'ip netns exec ce-a' says no such namespace	Namespace lost on reboot	Re-run namespace creation commands or run <code>/etc/rc.local</code>
CE-A cannot ping CE-B	VRF BGP routes not propagated	Check 'show bgp vrf vrf-mission' for routes on both PEs
veth-pe-cea not in vrf-mission	master not set	<code>sudo ip link set veth-pe-cea master vrf-mission</code>
Ping works but iPerf3 cannot connect	CE-B iPerf3 server not running	<code>ssh s2 'sudo ip netns exec ce-b iperf3 -s -D'</code>

IPSec Issues

Symptom	Cause	Fix
'ipsec status' shows 0 SAs	Tunnel not connecting	<code>sudo ipsec restart</code> on both; check <code>/var/log/charon.log</code>

Symptom	Cause	Fix
AUTHENTICATION_FAILURE in charon.log	Certificate mismatch	Regenerate certs; ensure CN matches leftid/rightid
Tunnel established but no traffic	Subnet mismatch	Check leftsubnet/rightsubnet match CE namespace subnets
Tunnel drops frequently	DPD timeout too aggressive	Increase dpdtimeout=300s in ipsec.conf

Static IP / Netplan Issues

Symptom	Cause	Fix
IP reverts to DHCP on reboot	cloud-init overwriting netplan	sudo touch /etc/cloud/cloud-init.disabled
netplan apply gives YAML error	Tab characters in YAML	Open file and replace all tabs with spaces (2 per indent)
eth0 not found	Interface name is different	Run 'ip link show' and find actual name; replace eth0
No internet after static IP	Missing default route	Ensure 'routes: - to: default via: 192.168.1.1' is in netplan

B Quick Command Reference

From Laptop -- Daily Use

Action	Command
SSH to station1	ssh s1
SSH to station2	ssh s2
SSH to station3	ssh s3
Ping all Pis	for H in 10 20 30; do ping -c 1 192.168.1.\$H; done
Run command on all Pis	for H in s1 s2 s3; do ssh \$H 'COMMAND'; done

FRRouting (run from vtysh)

What to Check	vysh Command
OSPF neighbors	show ip ospf neighbor
OSPF routes	show ip route ospf
BGP summary	show ip bgp summary
BGP VPN routes	show ip bgp vpnv4 all
MPLS LDP neighbors	show mpls ldp neighbor
MPLS LDP discovery	show mpls ldp discovery
MPLS label table	show mpls table
VRF BGP routes	show bgp vrf vrf-mission ipv4 unicast
All interfaces	show interface brief
Running config	show running-config
Save config	write memory
TE interface info	show mpls-te interface

IPSec

Action	Command
Check tunnel status	sudo ipsec status
Restart IPSec	sudo ipsec restart

Action	Command
Watch IKE log	sudo tail -f /var/log/charon.log
Check rekey events	sudo grep rekey /var/log/charon.log

Network Namespaces

Action	Command
List namespaces	ip netns list
Run command in CE-A	sudo ip netns exec ce-a COMMAND
Ping from CE-A to CE-B	sudo ip netns exec ce-a ping 10.100.2.1
iPerf3 from CE-A	sudo ip netns exec ce-a iperf3 -c 10.100.2.1 -u -b 10M -t 30
iPerf3 server in CE-B	sudo ip netns exec ce-b iperf3 -s

System

Action	Command
Check FRR status	sudo systemctl status frr
Restart FRR	sudo systemctl restart frr
Check NTP sync	chronyc tracking
Check MPLS on eth0	sysctl net.mpls.conf.eth0.input
Check VRFs	ip vrf show
Check kernel MPLS modules	lsmod grep mpls
Watch interface traffic	sudo nload eth0
Capture OSPF packets	sudo tcpdump -i eth0 proto ospf
Capture BGP packets	sudo tcpdump -i eth0 port 179

Fault Injection (for ML Training Data)

Fault Type	Injection Command	Remove Command
Congestion on eth0	sudo tc qdisc add dev eth0 root tbf rate 20mbit burst 32kbit latency 400ms	sudo tc qdisc del dev eth0 root
Packet loss 5%	sudo tc qdisc add dev eth0 root netem loss 5%	sudo tc qdisc del dev eth0 root

Fault Type	Injection Command	Remove Command
Latency 50ms	<code>sudo tc qdisc add dev eth0 root netem delay 50ms</code>	<code>sudo tc qdisc del dev eth0 root</code>
Jitter 10ms	<code>sudo tc qdisc add dev eth0 root netem delay 10ms 5ms</code>	<code>sudo tc qdisc del dev eth0 root</code>
BGP session clear	<code>sudo vtysh -c 'clear ip bgp 10.1.3.1 soft'</code>	Sessions re-establish automatically
VRF route leak	<code>sudo vtysh -c 'conf t'</code> then modify RT import	Remove the wrong RT import